

Visualization for machine learning (vis4ml)

CS424: Visualization & Visual Analytics
Fabio Miranda



urbantk.org



evl | electronic
visualization
laboratory



**UNIVERSITY OF
ILLINOIS CHICAGO**

ML → vis, now vis → ML

5134

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 28, NO. 12, DECEMBER 2022

A Survey on ML4VIS: Applying Machine Learning Advances to Data Visualization

Qianwen Wang[✉], Chen Zhu-Tian[✉], Yong Wang[✉], and Huamin Qu

Abstract—Inspired by the great success of machine learning (ML), researchers have applied ML techniques to visualizations to achieve a better design, development, and evaluation of visualizations. This branch of studies, known as ML4VIS, is gaining increasing research attention in recent years. To successfully adapt ML techniques for visualizations, a structured understanding of the integration of ML4VIS is needed. In this article, we systematically survey 88 ML4VIS studies, aiming to answer two motivating questions: “*what visualization processes can be assisted by ML?*” and “*how ML techniques can be used to solve visualization problems?*” This survey reveals seven main processes where the employment of ML techniques can benefit visualizations: *Data Processing4VIS*, *Data-VIS Mapping*, *Insight Communication*, *Style Imitation*, *VIS Interaction*, *VIS Reading*, and *User Profiling*. The seven processes are related to existing visualization theoretical models in an ML4VIS pipeline, aiming to illuminate the role of ML-assisted visualization in general visualizations. Meanwhile, the seven processes are mapped into main learning tasks in ML to align the capabilities of ML with the needs in visualization. Current practices and future opportunities of ML4VIS are discussed in the context of the ML4VIS pipeline and the ML-VIS mapping. While more studies are still needed in the area of ML4VIS, we hope this article can provide a stepping-stone for future exploration. A web-based interactive browser of this survey is available at <https://ml4vis.github.io>.

Index Terms—ML4VIS, machine learning, data visualization, survey

1 INTRODUCTION

DATA visualization (VIS), which maps data into visual presentations (e.g., position, color), is a powerful approach to uncover hidden insights and communicate compelling stories in data. However, achieving an effective visualization is typically challenging, requiring a large amount of human effort and having a high reliance on expertise, such as graphic design, user experience design, and data analysis.

Machine Learning (ML), on the other hand, provides a practical opportunity to relieve the reliance on experts in visualization. By automatically learning knowledge from data, ML enables task completion without explicit instructions from humans. The application of ML techniques can benefit a variety of visualization-related problems. In this paper, we define visualization-related problems as problems that are related to the process of creating, interacting with, and evaluating visualizations. We refer to studies that apply ML techniques to solve visualization-related problems using the knowledge extracted from data as ML4VIS in this paper. ML4VIS studies can be traced back to 1986 [89] and have been reignited by the recent advances in ML. A series of

ML4VIS studies are emerging, covering a wide range of visualization problems (e.g., graph layout [50], visualization assessment [54]) and ML techniques (e.g., variational auto-encoders [49], Mask-RCNN [60]).

While ML4VIS generates a stream of new opportunities, it also poses a series of challenges. First, it is not always clear, among various problems related to visualization, which one can be significantly improved by existing ML techniques and which one still requires a high level of human intervention. Roughly applying ML techniques to unsuitable visualization problems may only impose the drawbacks of ML (e.g., uncertainty, inexplicability) and threaten the validity of a visualization without bringing any benefit. Second, given a visualization-related task, selecting a proper ML technique and employing necessary adaptation are crucial yet challenging. There are a plethora of ML techniques, most of which are exclusively suitable for a certain type of problems. For example, to automatically interpret infographics, ML techniques developed for natural-image-understanding are more helpful and relevant than ML techniques developed for speech recognition. Therefore, the success of ML4VIS hinges on a better understanding of both the visualization and the ML, as well as the integration between the two.

Researchers have contributed a series of frameworks and surveys about the integration between visualization and ML, but they mainly investigate how visualization can assist ML in data analysis [90], [91], [92], [93], [94], [95] rather than how ML can be used to solve visualization-related problems. For example, in a state-of-the-art report, Ender et al. [92] depicted a closer integration between visualization and ML techniques through interactive ML. However, the goal of this integration is still to facilitate data analysis in different application domains (e.g., text analytics, multimedia analytics). Some recent surveys have reviewed

- Qianwen Wang is with Harvard University, Cambridge, MA 02138 USA. E-mail: qianwen_wang@hms.harvard.edu.
- Chen Zhu-Tian is with the University of California San Diego, La Jolla, CA 92093 USA. E-mail: zhutian@ucsd.edu.
- Yong Wang is with Singapore Management University, Singapore 188065, Singapore. E-mail: yongwang@smu.edu.sg.
- Huamin Qu is with the Hong Kong University of Science and Technology, Hong Kong. E-mail: huamin@cse.ust.hk.

Manuscript received 1 Dec. 2020; revised 8 Aug. 2021; accepted 16 Aug. 2021. Date of publication 26 Aug. 2021; date of current version 27 Oct. 2022. (Corresponding author: Qianwen Wang.) Recommended for acceptance by R. Maciejewski. Digital Object Identifier no. 10.1109/TVCG.2021.3106142

1077-2626 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on October 27, 2025 at 18:54:12 UTC from IEEE Xplore. Restrictions apply.

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 30, NO. 12, DECEMBER 2024

7637

Visual Analytics for Machine Learning: A Data Perspective Survey

Junpeng Wang[✉], Shixia Liu[✉], Fellow, IEEE, and Wei Zhang[✉]

(Survey Paper)

Abstract—The past decade has witnessed a plethora of works that leverage the power of visualization (VIS) to interpret machine learning (ML) models. The corresponding research topic, VIS4ML, keeps growing at a fast pace. To better organize the enormous works and shed light on the developing trend of VIS4ML, we provide a systematic review of these works through this survey. Since data quality greatly impacts the performance of ML models, our survey focuses specifically on summarizing VIS4ML works from the data perspective. First, we categorize the common data handled by ML models into five types, explain the unique features of each type, and highlight the corresponding ML models that are good at learning from them. Second, from the large number of VIS4ML works, we tease out six tasks that operate on these types of data (i.e., data-centric tasks) at different stages of the ML pipeline to understand, diagnose, and refine ML models. Lastly, by studying the distribution of 143 surveyed papers across the five data types, six data-centric tasks, and their intersections, we analyze the prospective research directions and envision future research trends.

Index Terms—Explainable AI, machine learning, taxonomy, VIS4ML, visual analytics, visualization.

1. INTRODUCTION

THE recent success of machine learning (ML), especially deep learning (DL) [1], [2], has received significant interest from researchers. ML has witnessed a general trend towards increasingly powerful models, however, often at the cost of being less and less interpretable. With growing concerns about the safety and reliability of ML models, their poor interpretability has started to prevent them from being adopted in many safety-critical applications, such as medical diagnosis [3], [4] and autonomous driving [5], [6]. To mitigate this problem, enormous visualization (VIS) efforts have been devoted to explainable artificial intelligence (XAI [7]) recently, e.g., perturbing input data to probe ML models' decision boundary [8], [9], training interpretable surrogates to mimic ML models' behavior [10], [11], externalizing intermediate data of ML models to open the black-boxes [12], [13]. These works constitute a new research

field, i.e., VIS4ML, and an increasing number of papers are being published in it. This survey targets to structurally review them and shed light on their growing trend.

In the meantime, there is a rising tendency of shifting ML models' developments from model-centric to data-centric [14]. Although we live in the era of Big Data, there are many quality issues rooted in the data, such as noisy labels [15], missing items [16], and imbalanced data distributions [17]. As the modeling techniques get more and more mature, it becomes increasingly obvious to ML developers that more performance gains could be achieved from the improvement of data rather than models. So, along with the fast evolution of ML models, improving data quality for ML models attracts more research attention recently [14]. This also echoes the famous proverb “Garbage In, Garbage Out”, i.e., we can never get a satisfactory ML model without quality input data. The shift towards data-centric modeling from the ML field has also inspired many pioneering VIS works on inspecting and improving data quality through data curation/correction/purification [15], [18], [19]. To promote this emerging direction, we revisit and structurally review existing VIS4ML works from a data perspective to disclose what efforts have been conducted and what opportunities remain open. Such a review will inspire more VIS4ML ideas and drive more data-oriented innovations.

Our data-centric survey aims to systematically review VIS4ML works by disclosing *what* data they have focused on and *how* the data have been operated to interpret, diagnose, and refine ML models. It is carried out from three aspects. First, we identify the most common *data types* processed by ML models, their unique features, and how ML models have been tailored to better learn from them (Section IV). Second, focusing on the operations applied to the data, we elicit six *data-centric VIS4ML tasks* serving the general goal of model understanding, diagnosis, and refinement [20], [21] (Section V). Third, by studying the distribution of the surveyed papers across different data types, VIS4ML tasks, and their intersections, we summarize the ongoing research trend and disclose prospective VIS4ML research directions (Section VI).

The contributions of our survey are twofold. First, we provide a data-centric taxonomy for VIS4ML and review papers following it. The taxonomy and review help better understand the growing number of VIS4ML works, reexamine them in a new angle, and unblock researchers from proposing more data-centric works. Second, from the coverage of the surveyed

Manuscript received 19 January 2023; revised 16 December 2023; accepted 15 January 2024. Date of publication 23 January 2024; date of current version 29 October 2024. Recommended for acceptance by A. Enderl. (Corresponding author: Junpeng Wang.) Junpeng Wang and Wei Zhang are with Visa Research, Foster City, CA 94404 USA (e-mail: junpeng.wang.nk@gmail.com; wzhan@visa.com). Shixia Liu is with Tsinghua University, Beijing 100084, China (e-mail: shixia@tsinghua.edu.cn). Digital Object Identifier 10.1109/TVCG.2024.3357065

1077-2626 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Understanding the gap

- Model understanding has yet to catch up to model performance improvements.
- Visualization is key to improve model understanding to a wide range of audiences.
- There are many rich domains to apply vis for machine learning techniques, including industry and research.

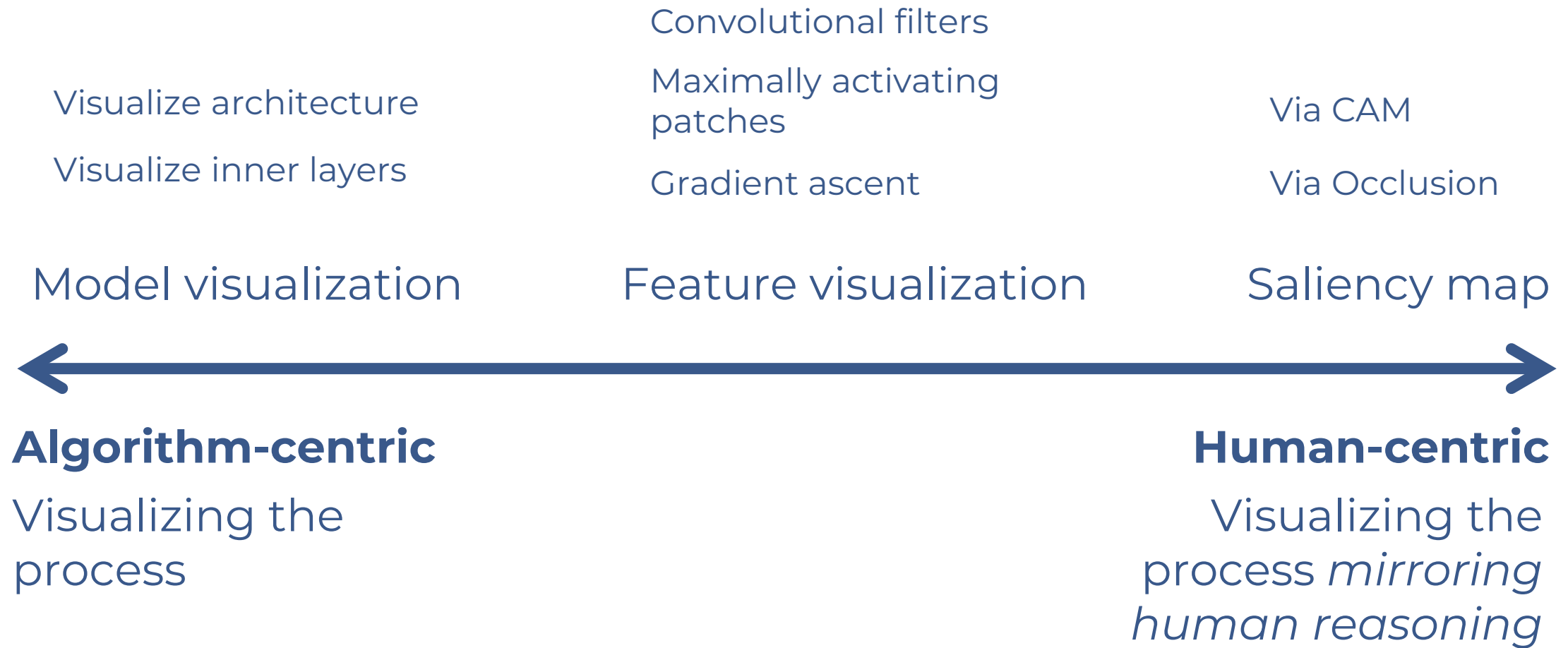
Explainable AI

- Why?
- What?
- How?

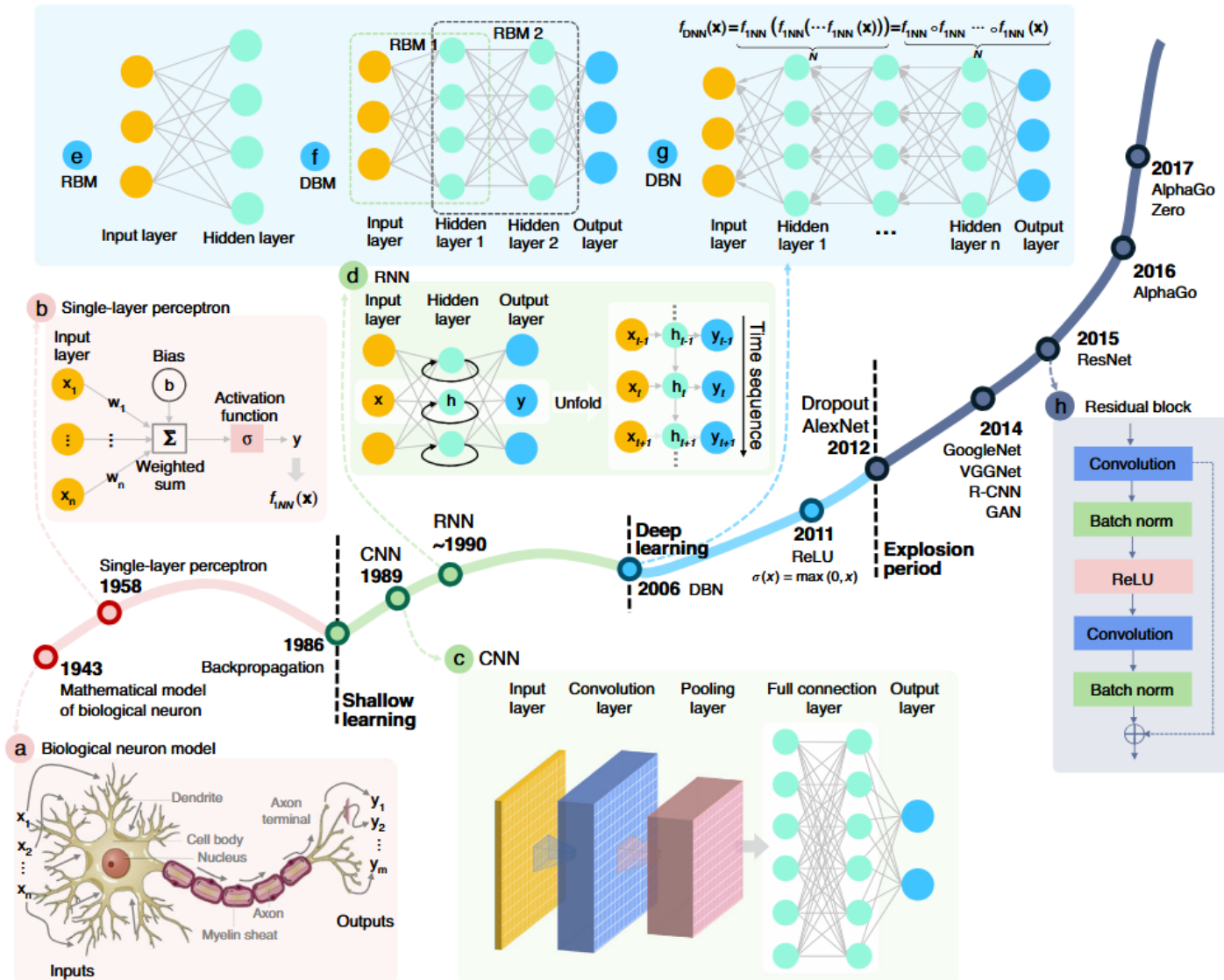
Explainable AI

- Why?
 - AI models behave like a “black box”: we see the output not the reasoning.
 - Build trust: People are more likely to use AI when they understand its decisions.
 - Ensure fairness: Explanations help detect bias and harmful behavior.
- What?
 - Methods and tools that make AI decisions **understandable to humans**.
 - Answers questions like: “**Why did the model predict this?**” and “**What would change the decision?**”
- How? Next slide!

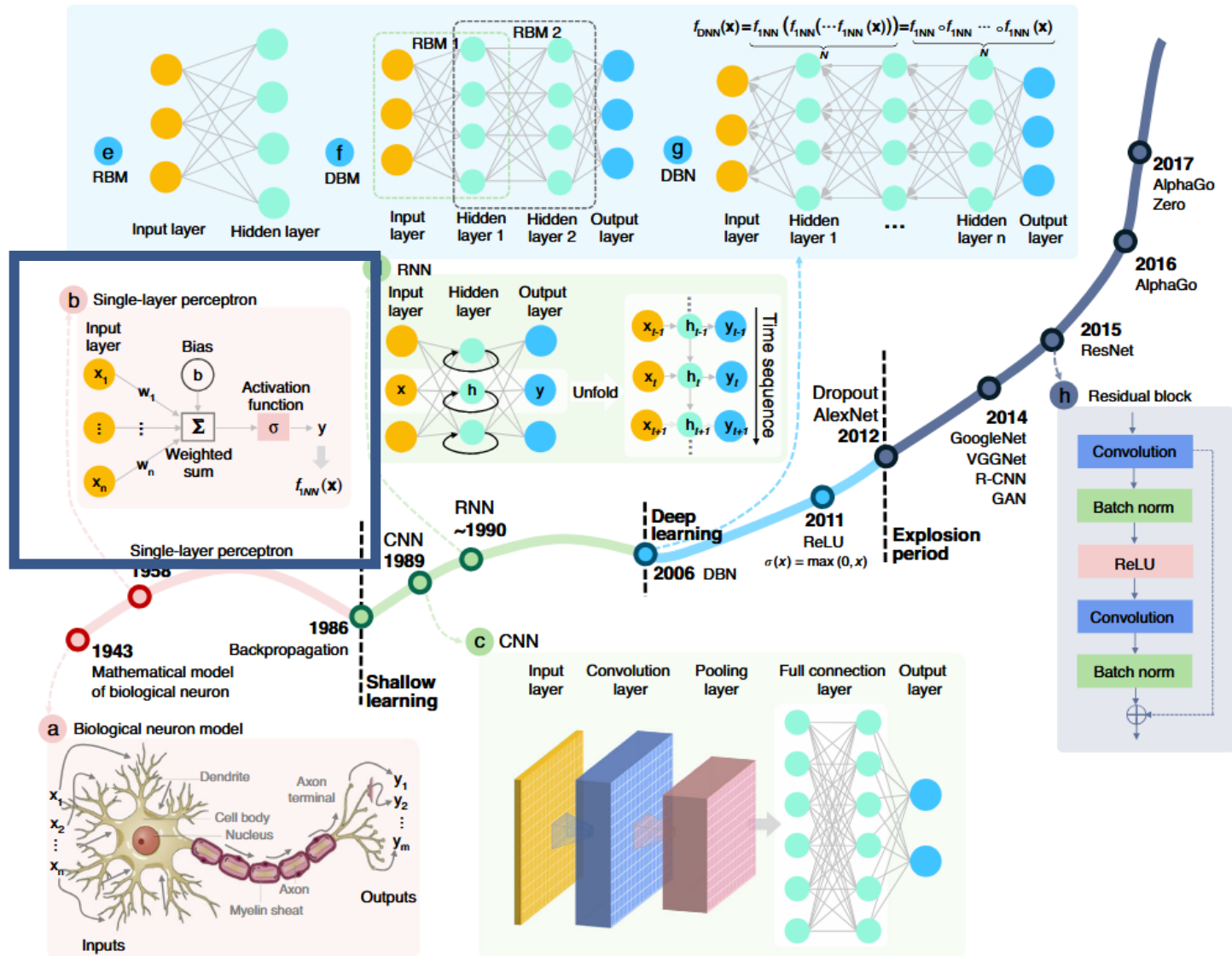
Spectrum of visualization for ML



Artificial neural networks



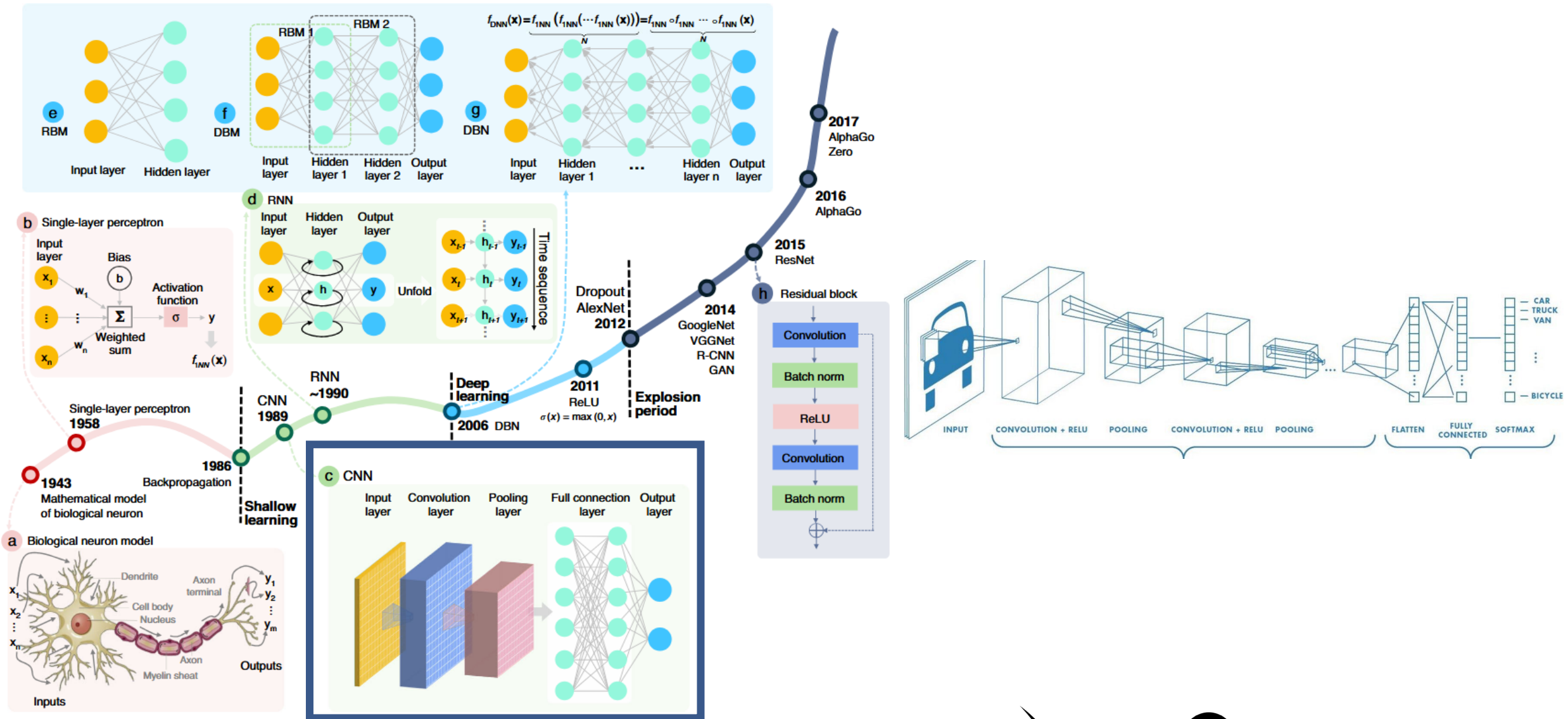
Artificial neural networks



Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

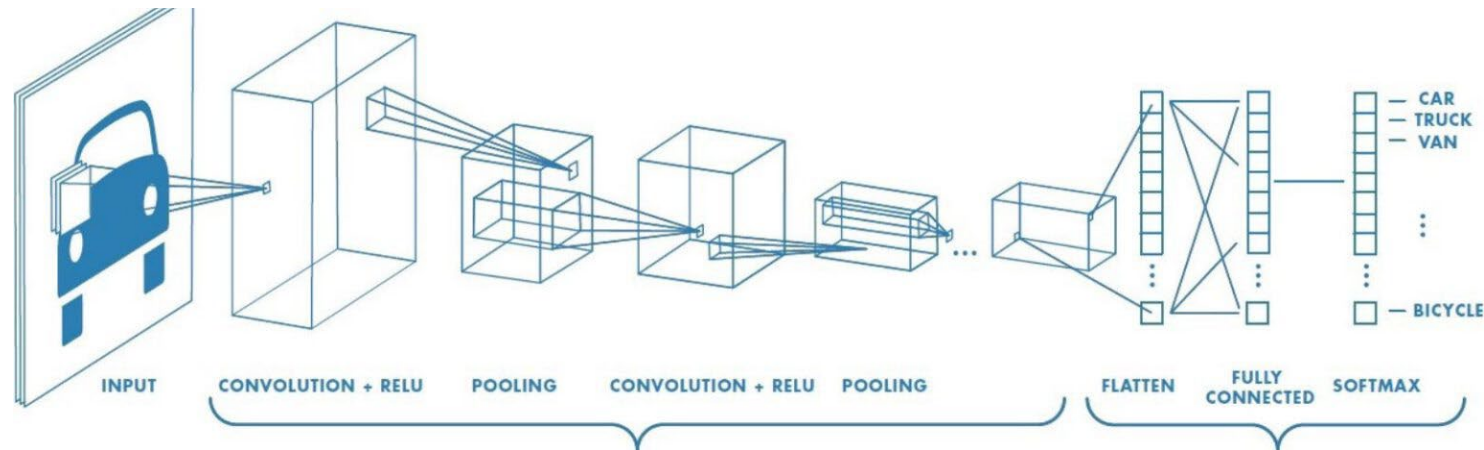
Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

Artificial neural networks



From: "Deep learning in optical metrology: a review"

Convolutional neural networks

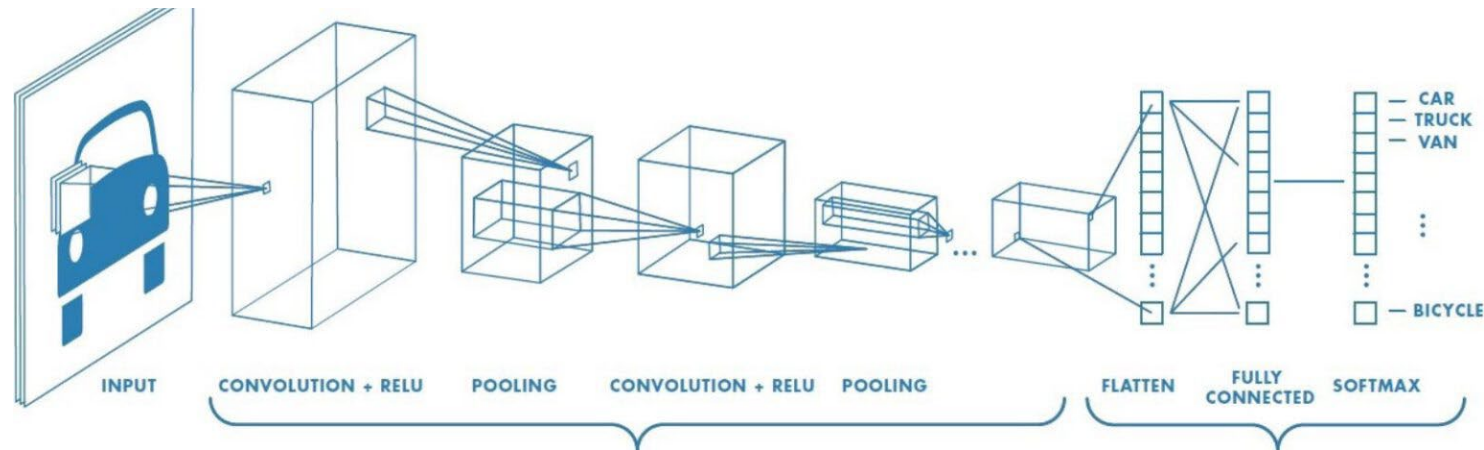


What is a convolutional layer?

1. Takes an input (e.g., image)
2. Has a kernel or filter
3. Slides this kernel over the image
4. At each position, it multiplies and adds to produce one output pixel.

$$\text{output}(x, y) = \sum_{i, j} \text{kernel}(i, j) \cdot \text{input}(x + i, y + j)$$

Convolutional neural networks

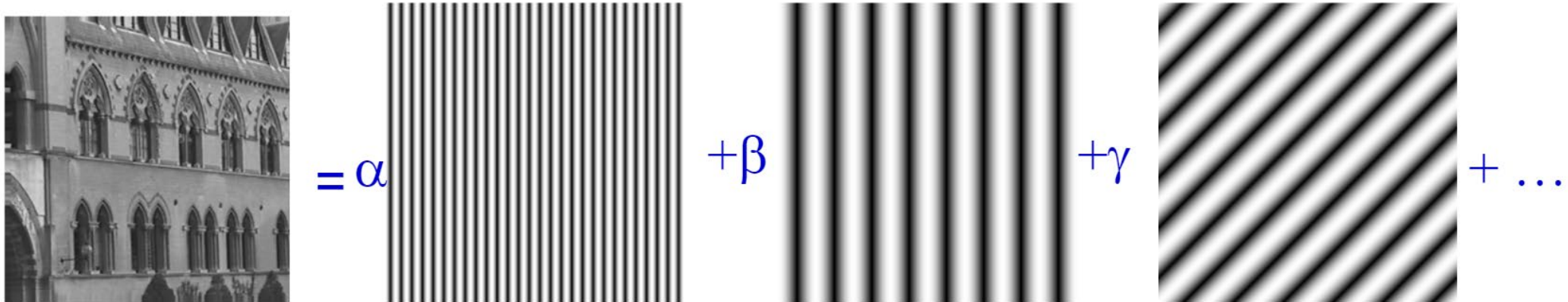


Convolution theorem:

Convolution in the spatial domain $\leftrightarrow$ Multiplication in the frequency domain.

Convolutional neural networks

$f(x,y)$



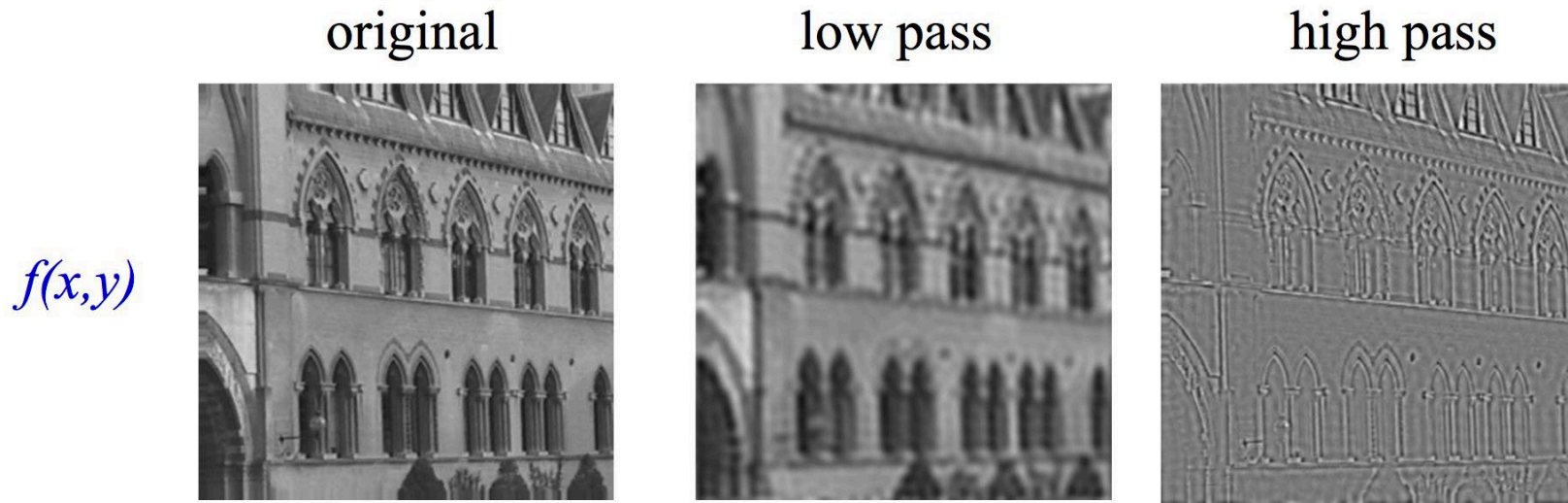
Convolution theorem:

Convolution in the spatial domain $\leftrightarrow$ Multiplication in the frequency domain.

So applying a convolution kernel to an image is equivalent to:

1. Transforming the image and kernel to the frequency domain
2. Multiplying their frequency representations

Convolutional neural networks



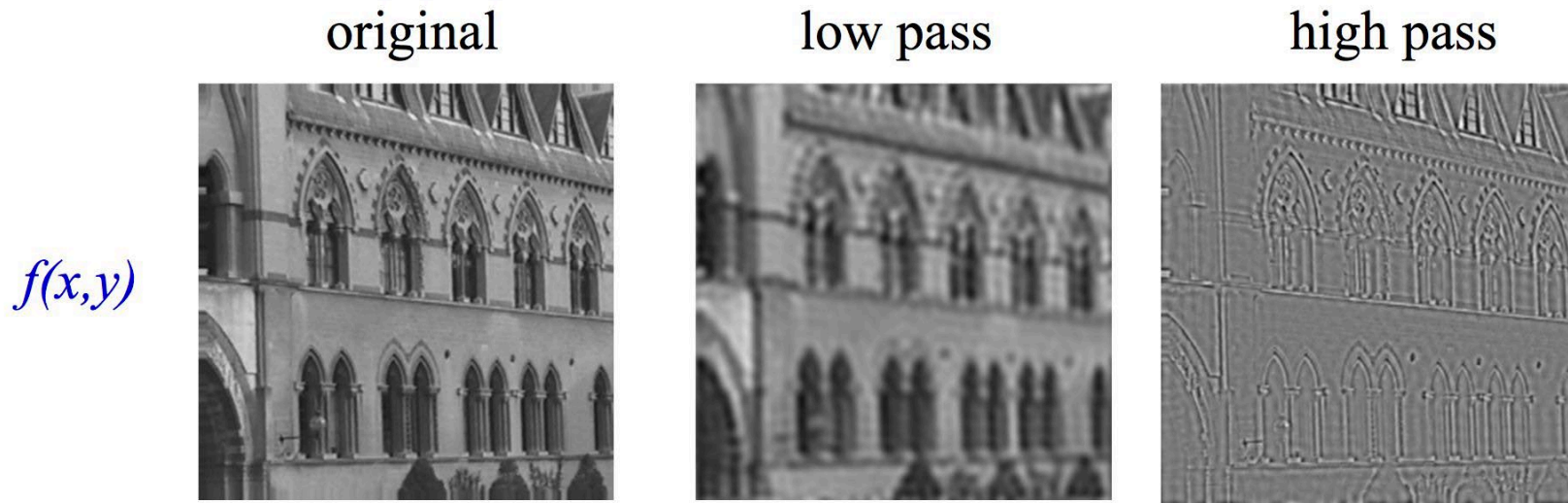
A **low-pass filter** such as a simple blur kernel in 2D is:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

What is happening?

- Each output pixel becomes the **average of its 3×3 neighborhood**.
- This averages away sharp changes → edges get softened → image looks blurrier.
- In frequency terms: high frequencies are reduced → low-pass behavior.

Convolutional neural networks



A **high-pass filter** such as a simple **Laplacian-type kernel** is:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

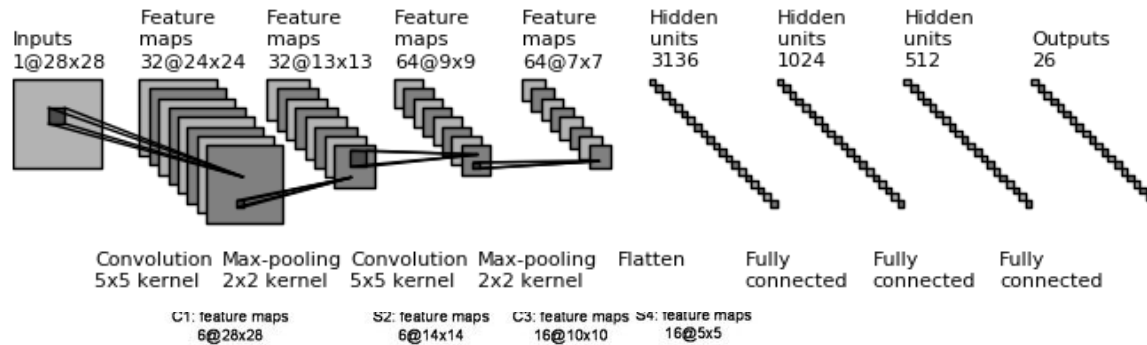
What is happening?

- The center pixel is multiplied by 4.
- The neighbors (up, down, left, right) are subtracted.
- So if the center is very different from its neighbors $\rightarrow$ large value (edge)..

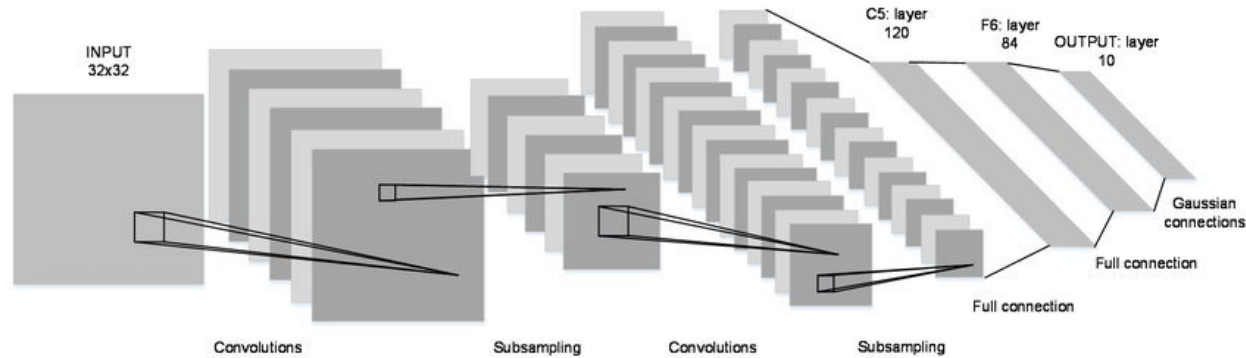
Convolutional neural networks

- What does this mean for CNNs?
- Some learned kernels end up behaving like high-pass filters:
 - They active strongly on edges and texture.
 - They help the network detect important boundaries and patterns in the image.
- Once a convolutional layer has produced these feature maps (e.g., maps that light up on edges), a pooling layer:
 - Looks at small regions for each feature map.
 - Applies a simple rule like max or average to each block.
 - Produces a smaller feature map that keeps the most important information.

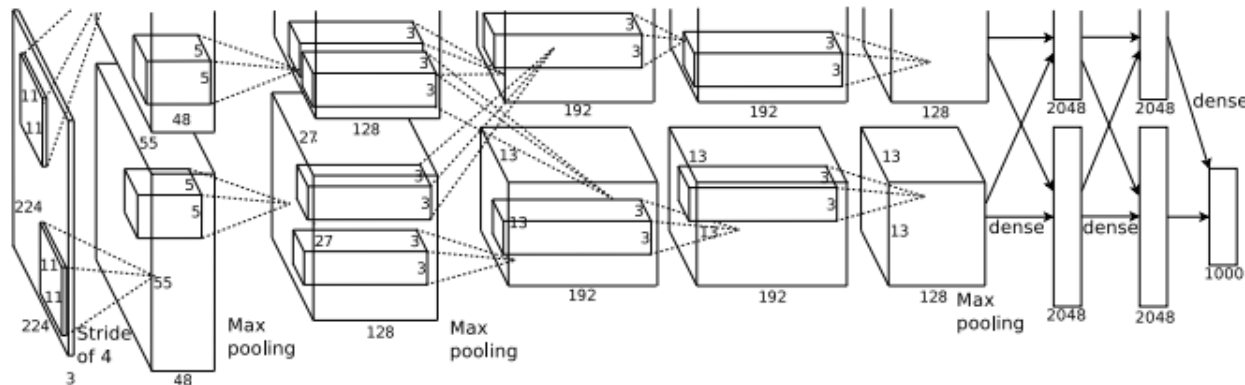
Convolutional neural networks



ConvNet in 1989



LeNet in 1998

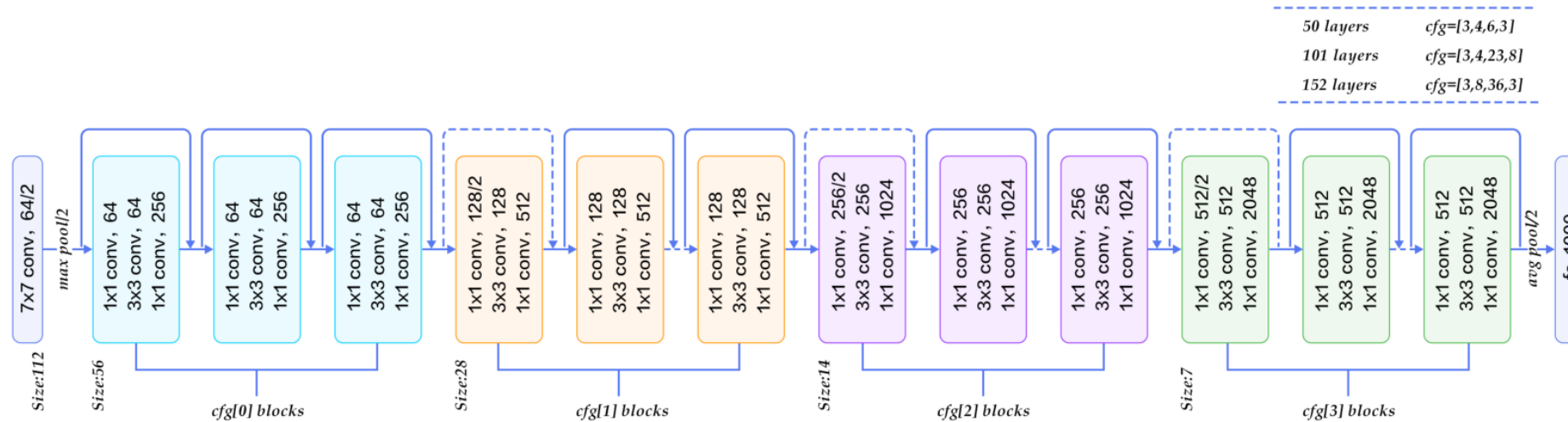


AlexNet in 2012

Convolutional neural networks

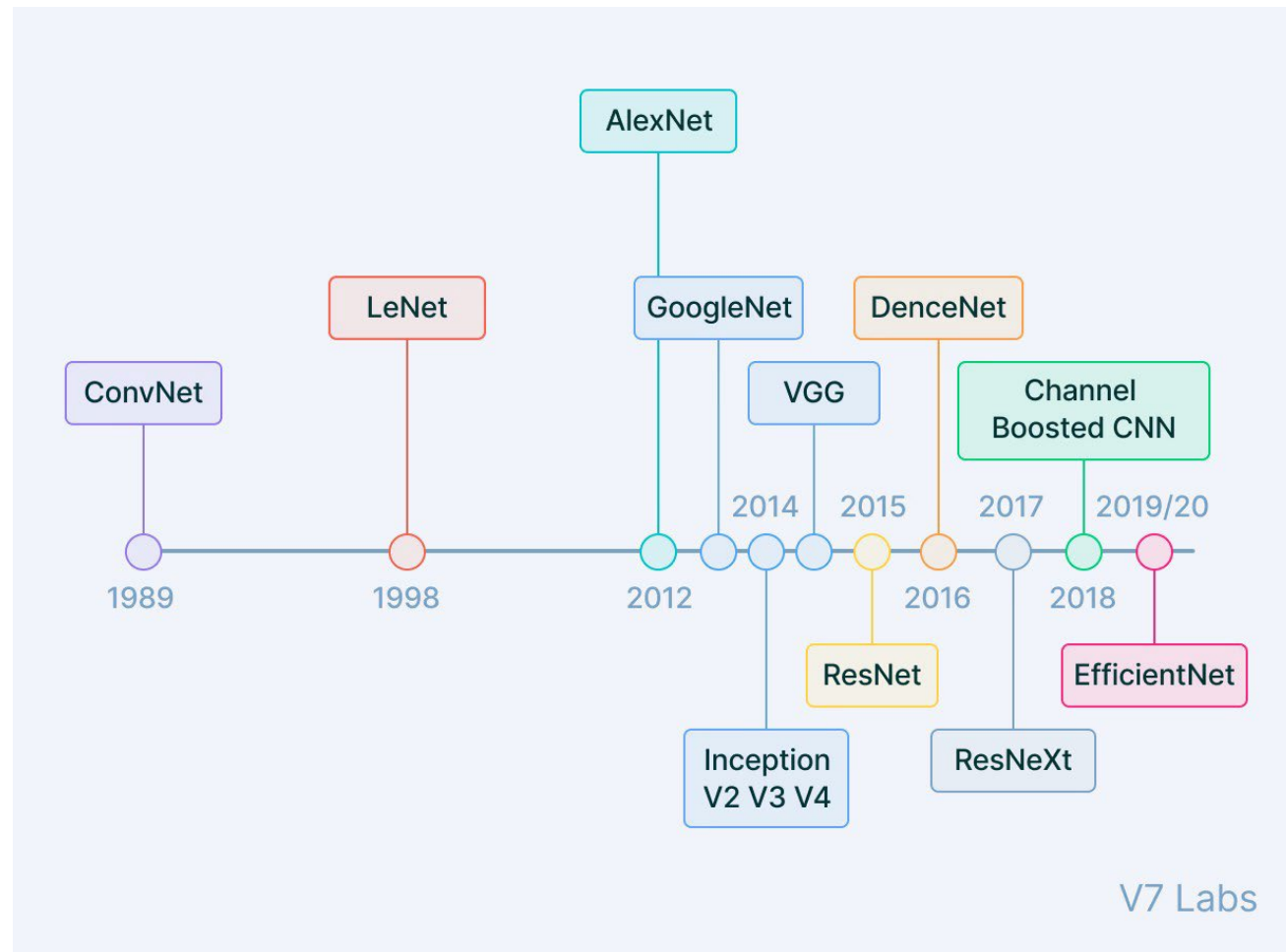


VGG in 2014



ResNet in 2015

Convolutional neural networks



From: V7 labs

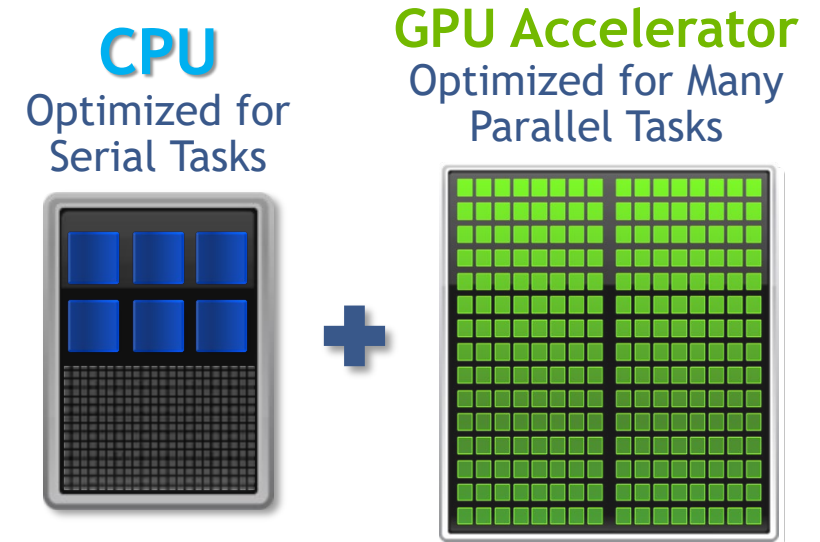
Convolutional neural networks

Progress in ML and AI is driven by data,
computation resources, and AI algorithm
innovations.

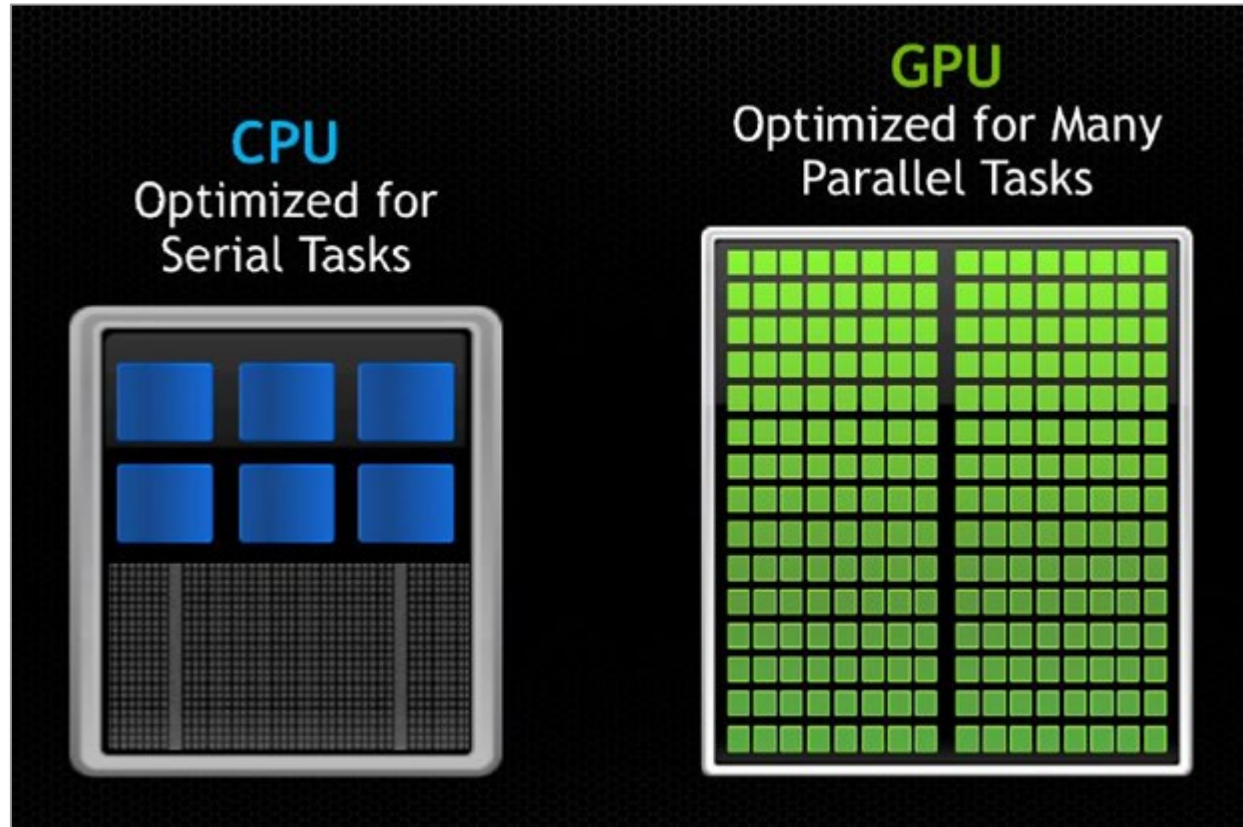


GPUs

- GPUs are designed to perform high-speed parallel calculations for real-time rendering (embarrassingly parallel task).
- 10-100x speed-ups over CPUs when applied in GPGPU.
- Why?
 - CPU contains few powerful cores, GPU contains hundreds of smaller cores.
 - CPU: individual threads execute instructions independently (SISD). GPU: single instruction, multiple threads (SIMT).
 - Shared memory for algorithms with a high degree of locality.



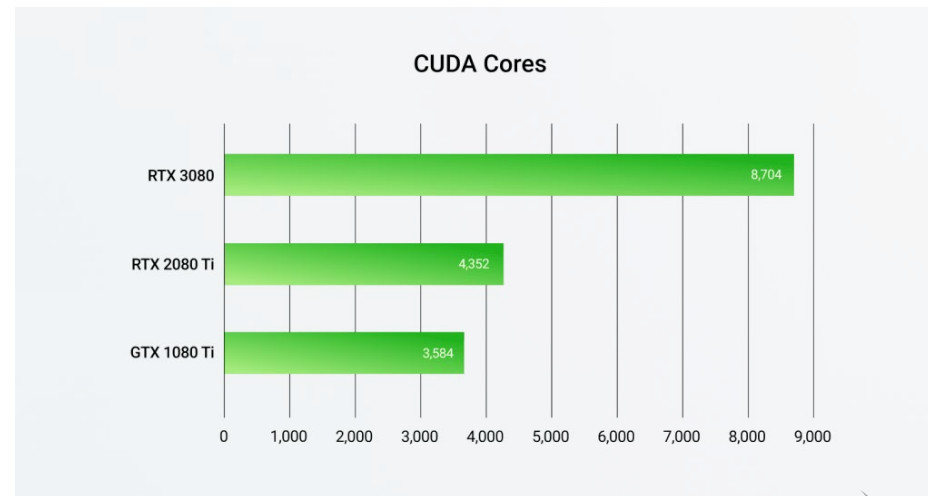
Modern GPUs





GPU architecture

- Global memory:
 - Similar to CPU's RAM.
 - Accessible by both CPU and GPU.
 - Limited: < 24 GB
- Streaming multiprocessors (SMs)
 - Perform the actual computations.
 - Each SM has its own control units, registers, caches, **execution pipeline**.
 - 3080 RTX: 68 SMs, each with 128 CUDA cores.



Heterogeneous computing

- Host: CPU and its memory.
- Device: GPU and its memory.

```
texture<float, 2, cudaReadModeElementType> tex;

void foo()
{
    cudaArray* cu_array;

    // Allocate array
    cudaChannelFormatDesc description = cudaCreateChannelDesc<float>();
    cudaMallocArray(&cu_array, &description, width, height);

    // Copy image data to array
    cudaMemcpyToArray(cu_array, image, width*height*sizeof(float), cudaMemcpyHostToDevice);

    // Set texture parameters (default)
    tex.addressMode[0] = cudaAddressModeClamp;
    tex.addressMode[1] = cudaAddressModeClamp;
    tex.filterMode = cudaFilterModePoint;
    tex.normalized = false; // do not normalize coordinates

    // Bind the array to the texture
    cudaBindTextureToArray(tex, cu_array);

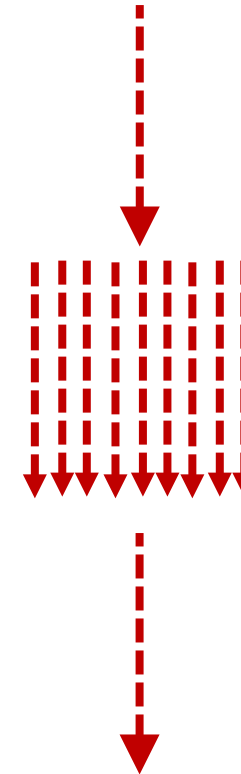
    // Run kernel
    dim3 blockDim(16, 16, 1);
    dim3 gridDim((width + blockDim.x - 1) / blockDim.x, (height + blockDim.y - 1) / blockDim.y, 1);
    kernel<<< gridDim, blockDim, 0 >>>(d_data, height, width);

    // Unbind the array from the texture
    cudaUnbindTexture(tex);
} //end foo()

__global__ void kernel(float* odata, int height, int width)
{
    unsigned int x = blockIdx.x*blockDim.x + threadIdx.x;
    unsigned int y = blockIdx.y*blockDim.y + threadIdx.y;
    if (x < width && y < height) {
        float c = tex2D(tex, x, y);
        odata[y*width+x] = c;
    }
}
```

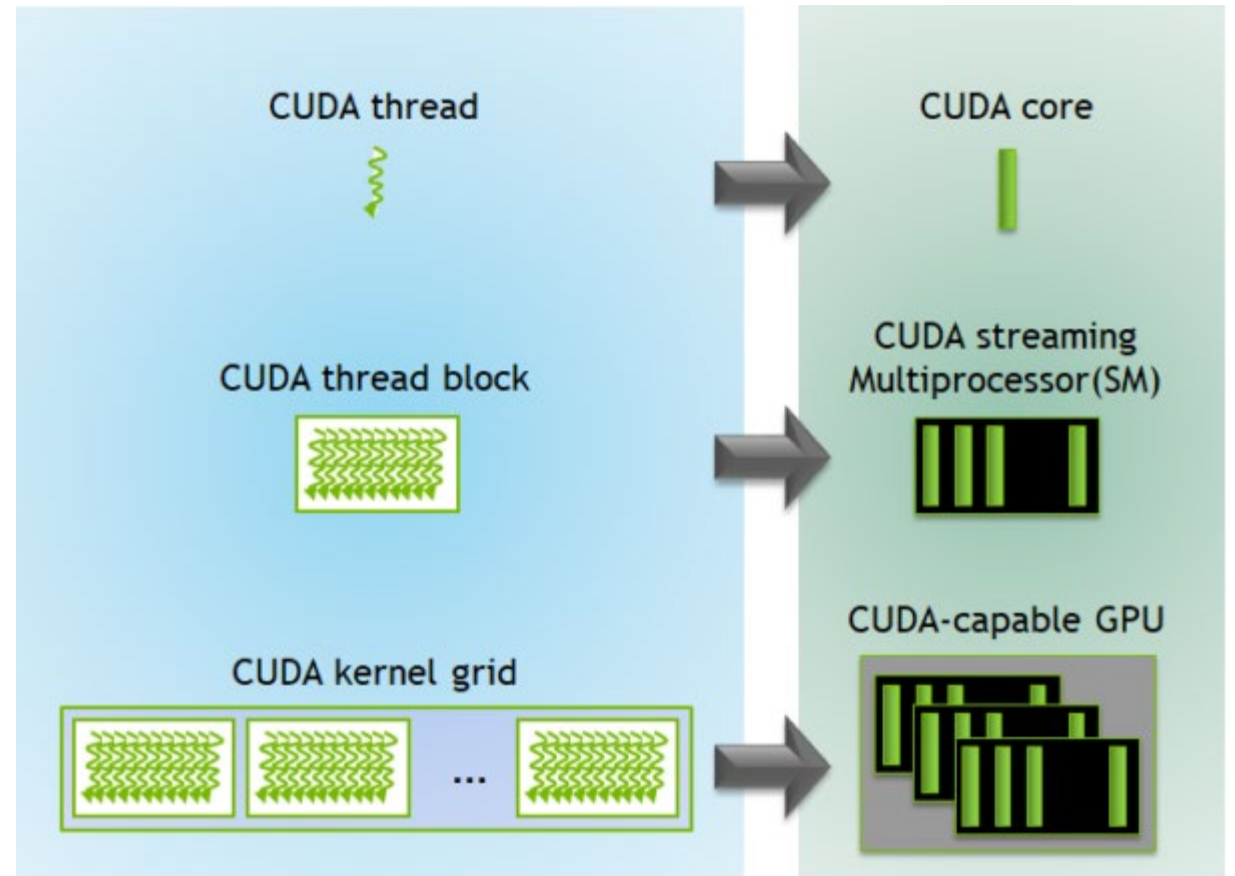
CPU

GPU



Processing flow

1. Copy input data from CPU to GPU memory.
2. Load GPU program and execute.
 - Group of threads is called a CUDA block, executed by one streaming multiprocessor (SM).
 - Set of blocks is referred to as a grid.
3. Copy results from GPU memory to CPU memory.



CUDA: vector addition

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main( int argc, char* argv[] )
{
    // Size of vectors
    int n = 100000;
    // Host input vectors
    double *h_a, *h_b;
    // Host output vector
    double *h_c;
    // Device input vectors
    double *d_a, *d_b;
    // Device output vector
    double *d_c;
    // Size, in bytes, of each vector
    size_t bytes = n*sizeof(double);
    // Allocate memory for each vector on host
    h_a = (double*)malloc(bytes); h_b = (double*)malloc(bytes); h_c = (double*)malloc(bytes);

    // Allocate memory for each vector on GPU
    cudaMalloc(&d_a, bytes); cudaMalloc(&d_b, bytes); cudaMalloc(&d_c, bytes);
```

Allocating memory
on the host and
device



CUDA: vector addition

```
// Initialize vectors on host
for( int i = 0; i < n; i++ ) {
    h_a[i] = sin(i)*sin(i);
    h_b[i] = cos(i)*cos(i);
}
```

```
// Copy host vectors to device
cudaMemcpy( d_a, h_a, bytes, cudaMemcpyHostToDevice);
cudaMemcpy( d_b, h_b, bytes, cudaMemcpyHostToDevice);
```

```
int blockSize, gridSize;
```

```
// Number of threads in each thread block
blockSize = 1024;
```

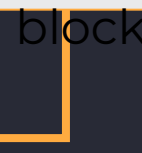
```
// Number of thread blocks in grid
gridSize = (int)ceil((float)n/blockSize);
```

```
// Execute the kernel
vecAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
```

Copying to device



Executing the
kernel, with 1024
threads per thread
block



CUDA: vector addition

```
// CUDA kernel. Each thread takes care of one element of c
__global__ void vecAdd(double *a, double *b, double *c, int n)
{
    // Get our global thread ID
    int id = blockIdx.x*blockDim.x+threadIdx.x;

    // Make sure we do not go out of bounds
    if (id < n)
        c[id] = a[id] + b[id];
}
```

CUDA kernel, runs on
device



From: <https://www.olcf.ornl.gov/tutorials/cuda-vector-addition/>

CUDA: vector addition

Copying from
device to host

```
// Copy array back to host
cudaMemcpy( h_c, d_c, bytes, cudaMemcpyDeviceToHost );

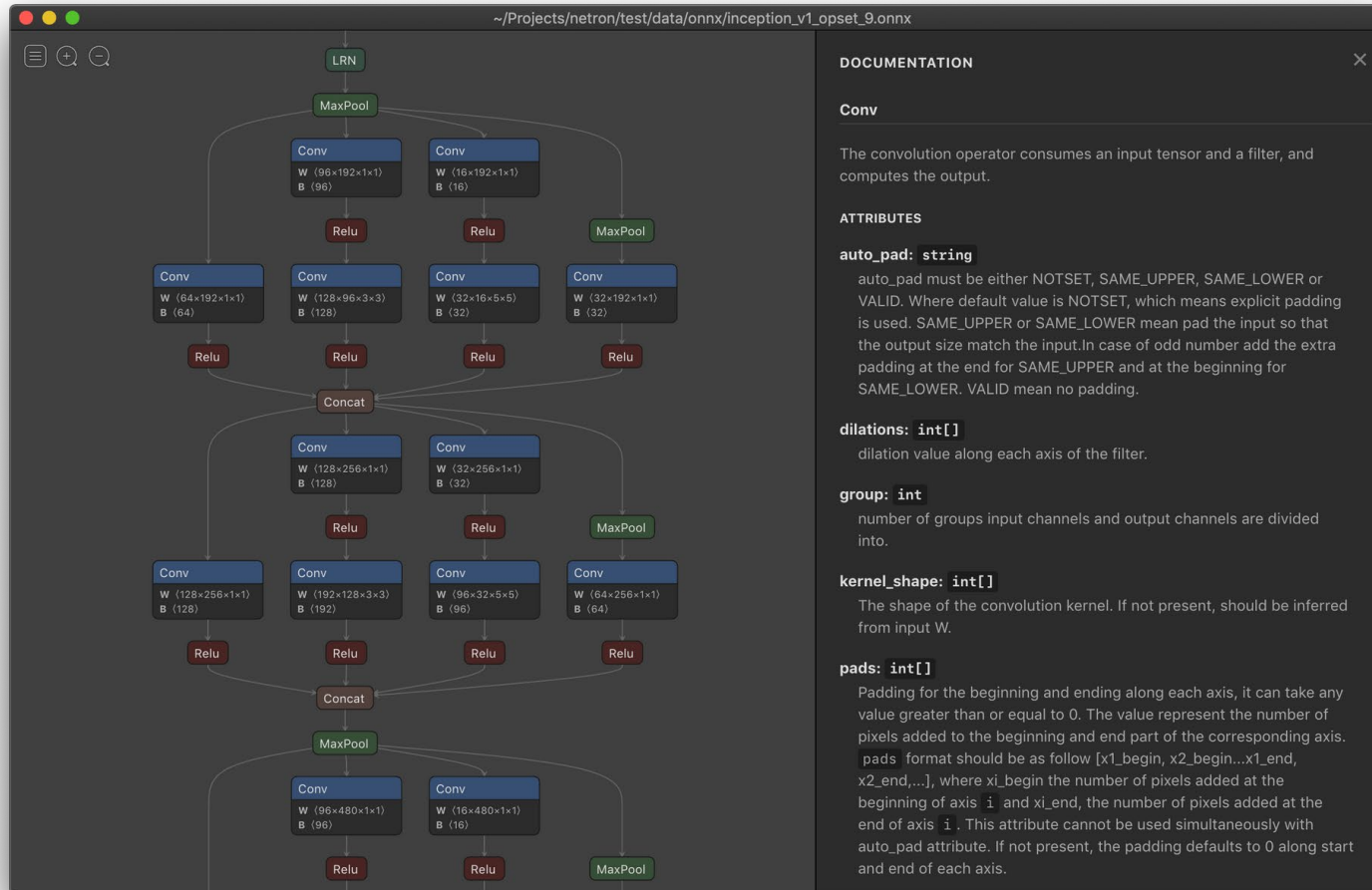
// Sum up vector c and print result divided by n, this should equal 1 within error
double sum = 0;
for(i=0; i<n; i++)
    sum += h_c[i];
printf("final result: %f\n", sum/n);

// Release device memory
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

// Release host memory
free(h_a);
free(h_b);
free(h_c);

return 0;
}
```

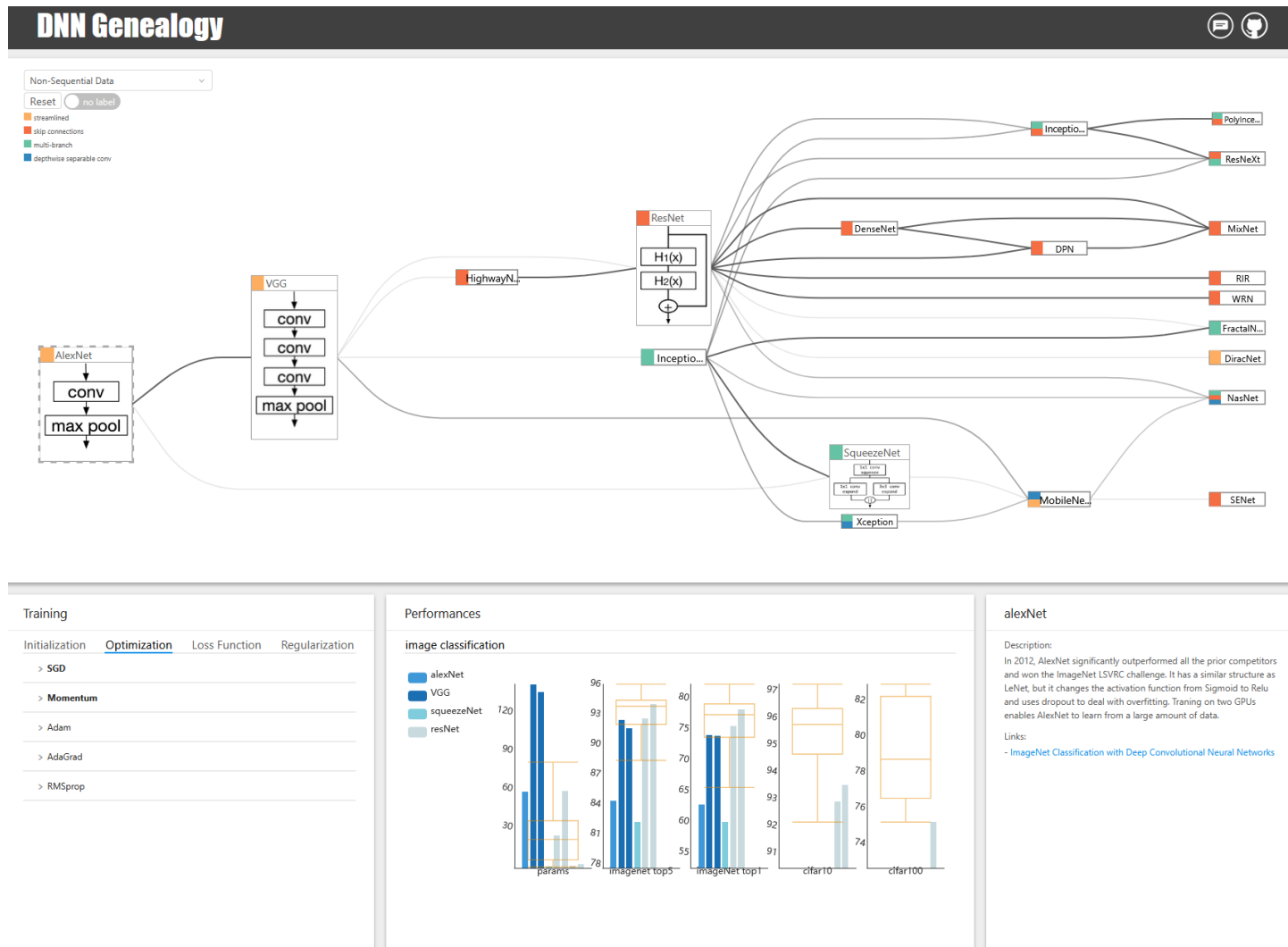
Visualizing ML architectures



<https://netron.app/?url=https://github.com/onnx/models/raw/main/validated/vision/classification/squeezenet/model/squeezenet1.0-3.onnx>



Visualizing ML architectures



<https://qianwen.info/demos/DNNGenealogy/>

VISUAL GENEALOGY OF DEEP NEURAL NETWORKS

Visual Genealogy of Deep Neural Networks

Qianwen Wang, Jun Yuan, Shuxin Chen, Hang Su, Huamin Qu, and Shixia Liu

Abstract—A comprehensive and comprehensible summary of existing deep neural networks (DNNs) helps practitioners understand the behaviour and evolution of DNNs, offers insights for architecture optimization, and sheds light on the working mechanisms of DNNs. However, this summary is hard to obtain because of the complexity and diversity of DNN architectures. To address this issue, we develop DNN Genealogy, an interactive visualization tool, to offer a visual summary of representative DNNs and their evolutionary relationships. DNN Genealogy enables users to learn DNNs from multiple aspects, including architecture, performance, and evolutionary relationships. Central to this tool is a systematic analysis and visualization of 66 representative DNNs based on our analysis of 140 papers. A directed acyclic graph is used to illustrate the evolutionary relationships among these DNNs and highlight the representative DNNs. A focus + context visualization is developed to orient users during their exploration. A set of network glyphs is used in the graph to facilitate the understanding and comparing of DNNs in the context of the evolution. Case studies demonstrate that DNN Genealogy provides helpful guidance in understanding, applying, and optimizing DNNs. DNN Genealogy is extensible and will continue to be updated to reflect future advances in DNNs.

Index Terms—Interactive Visual Summary, Information Visualization, Educational Tool, Deep Neural Networks.

1 INTRODUCTION

OVER the past few years, deep neural networks (DNNs) have exhibited state-of-the-art performances in many applications, including classification [59], language translation [44], [65], and speech recognition [5], [24]. As a breakthrough in artificial intelligence, DNNs have been attracting attention not only from researchers, but also from practitioners who are interested in applying DNNs to their applications.

Even though DNNs have achieved impressive progress, their working mechanisms remain unclear, especially to average practitioners. Principled rules for the design of DNNs (what architecture is the best choice for a specific task) remain lacking. Researchers keep exploring new DNN architectures on the basis of their empirical knowledge, expertise, and extensive experiments, thereby leading to the wide high diversity of existing DNNs. Practitioners, especially new beginners, can be easily baffled by the variety of available DNNs and often fail to understand them and apply them as needed. Therefore, an educational tool that provides informative guidance for understanding and applying DNNs is necessary.

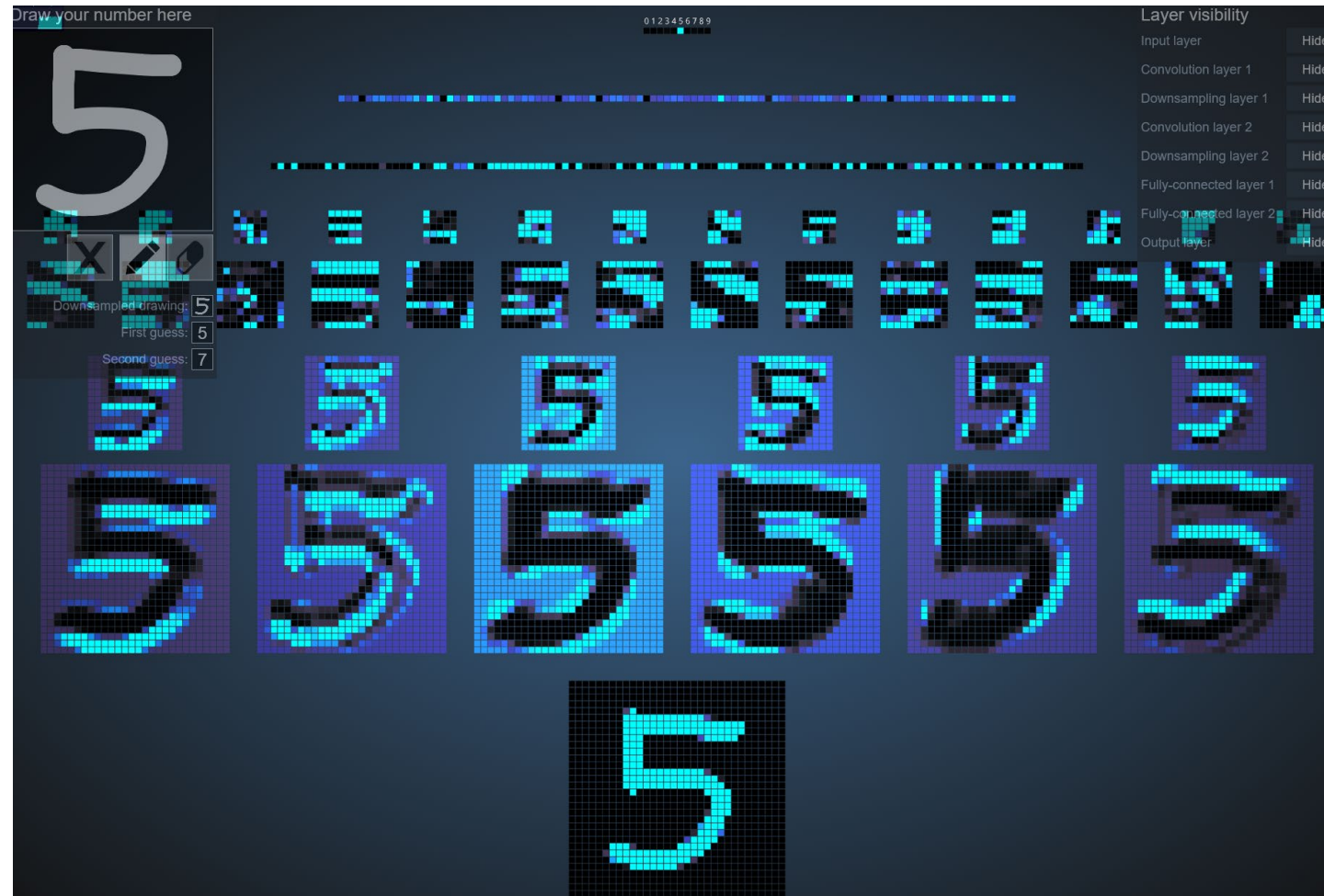
To inspire and motivate the wide adoption and extensive use of DNNs, a summary and presentation of existing DNNs are needed. However, summarizing DNNs and representing this summary are technically demanding. Three main challenges need to be addressed. The first challenge is caused by the **rising number of DNNs**. Due to their excellent performances, an increasing number of DNNs have been developed for different applications [6]. It is non-trivial to examine the numerous existing DNNs and summarize representative DNNs from them. Moreover, informative visual guidance is needed to prevent users from being overwhelmed when exploring these DNNs and thus make informed decisions when choosing which DNN to use for their tasks. The second challenge

comes from the **complexity of DNN architectures**. Advances in DNNs have led to complex architecture designs, including deep layers (over 1200 layers [28]), multiple branches [34], and dense skip connections [27]. Such complex architectures render the observation, comparison, and understanding of DNNs difficult, thus posing unique challenges for representation. The third challenge lies in the **diversity of DNNs**. Given the unclear working mechanisms of DNNs, DNN researchers and practitioners keep exploring various DNNs for different tasks, leading to a variety of DNN architectures. The diversity of DNN architectures results in challenges in identifying the evolutionary relationships among DNNs (one DNN is inspired by another DNN) and in revealing the evolution patterns of DNNs (DNN architectures are getting deeper).

To tackle the aforementioned challenges, we design and implement DNN Genealogy, a visual educational tool, to facilitate the exploratory analysis of different DNNs and to understand the pros and cons of each DNN in the context of model architecture and performance. To summarize the large number of existing DNNs, we employ a semi-automatic survey approach to extracting representative papers from a total of 16465 papers, constituting all papers published in CVPR, ICLR, NIPS, UAI, ICML, AAAI, and JMLR from 2012 to 2018. A hierarchical topic model [10] is employed to aid in the analysis of such massive literature. A four-step pipeline is developed to extract 140 relevant papers, from which we manually identify 66 representative DNNs. Consulting other surveys [21], [35], [67] and discussing with DNN experts further validate our summary. To assist in understanding and learning DNNs, we design and implement DNN Genealogy to present our summary and analysis of these 66 DNNs, as shown in Fig. 1. The evolutionary relationships among the DNNs are illustrated by a directed acyclic graph (DAG), where textual annotations provide detailed explanations of these relationships. A focus+context visualization based on a degree of interest (DOI) algorithm is developed to facilitate the exploration of these DNNs by directing users to relevant content. To help users quickly learn the main characteristic of a complex DNN architecture, we design

- Q. Wang and H. Qu are with Hong Kong University of Science and Technology. Email: {qiangbb@connect.ust.hk, huamin@cse.ust.hk}
- J. Yuan, S. Chen, H. Su, and S. Liu are with Tsinghua University. Email: {yuan-j15, csx15}@mails.tsinghua.edu.cn, {suhsangcs, shixia}@tsinghua.edu.cn

Visualizing inner layers



https://adamharley.com/nn_vis/cnn/2d.html

Visualizing features

What has the convolutional layer learned?

You have a trained neural network (e.g., CNN for images), you want to answer: “What pattern in a *real image* makes this filter fire?”

- Run a **forward pass** on many real images.
- Pick a feature map and the locations where its activation is high.

Visualization for machine learning

Visualizing and Understanding Convolutional Networks

Matthew D. Zeiler

Dept. of Computer Science, Courant Institute, New York University

ZEILER@CS.NYU.EDU

Rob Fergus

Dept. of Computer Science, Courant Institute, New York University

FERGUS@CS.NYU.EDU

Abstract

Large Convolutional Network models have recently demonstrated impressive classification performance on the ImageNet benchmark (Krizhevsky et al., 2012). However there is no clear understanding of why they perform so well, or how they might be improved. In this paper we address both issues. We introduce a novel visualization technique that gives insight into the function of intermediate feature layers and the operation of the classifier. Used in a diagnostic role, these visualizations allow us to find model architectures that outperform Krizhevsky *et al.* on the ImageNet classification benchmark. We also perform an ablation study to discover the performance contribution from different model layers. We show our ImageNet model generalizes well to other datasets: when the softmax classifier is retrained, it convincingly beats the current state-of-the-art results on Caltech-101 and Caltech-256 datasets.

1. Introduction

Since their introduction by (LeCun et al., 1989) in the early 1990's, Convolutional Networks (convnets) have demonstrated excellent performance at tasks such as hand-written digit classification and face detection. In the last year, several papers have shown that they can also deliver outstanding performance on more challenging visual classification tasks. (Ciresan et al., 2012) demonstrate state-of-the-art performance on NORB and CIFAR-10 datasets. Most notably, (Krizhevsky et al., 2012) show record beating performance on the ImageNet 2012 classification benchmark, with their convnet model achieving an error rate of 16.4%, compared to the 2nd place result of 26.1%. Several factors are responsible for this renewed inter-

est in convnet models: (i) the availability of much larger training sets, with millions of labeled examples; (ii) powerful GPU implementations, making the training of very large models practical and (iii) better model regularization strategies, such as Dropout (Hinton et al., 2012).

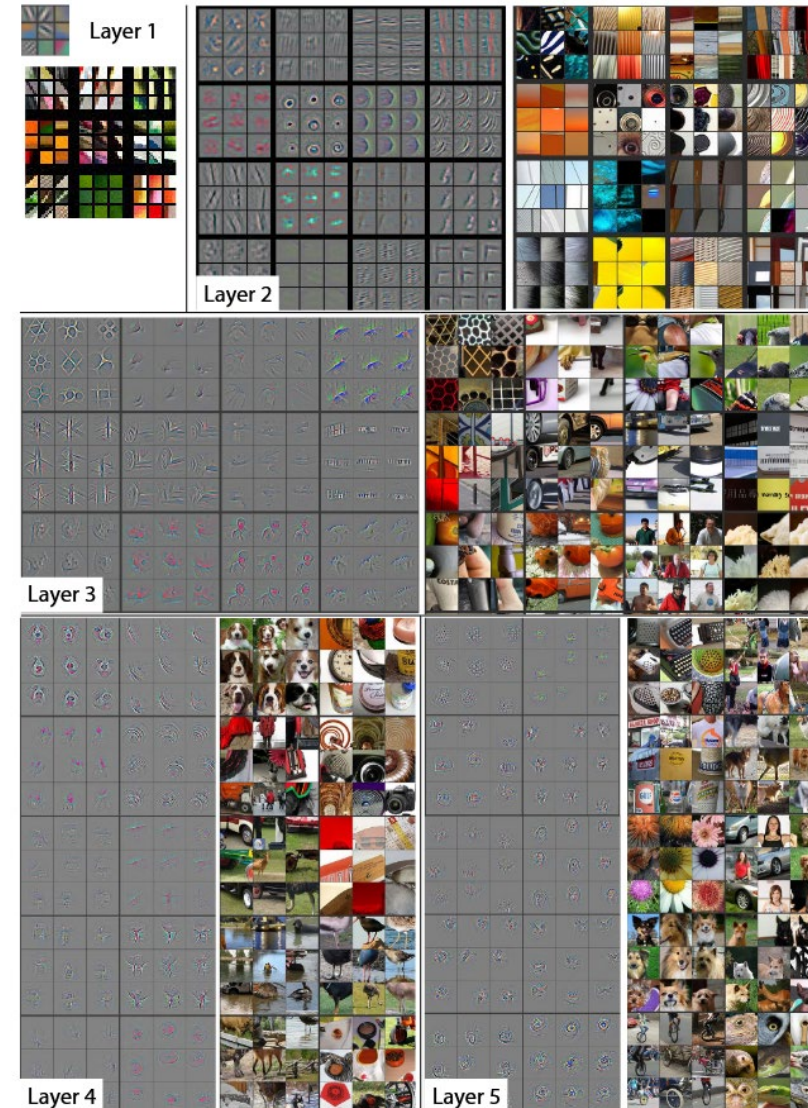
Despite this encouraging progress, there is still little insight into the internal operation and behavior of these complex models, or how they achieve such good performance. From a scientific standpoint, this is deeply unsatisfactory. Without clear understanding of how and why they work, the development of better models is reduced to trial-and-error. In this paper we introduce a visualization technique that reveals the input stimuli that excite individual feature maps at any layer in the model. It also allows us to observe the evolution of features during training and to diagnose potential problems with the model. The visualization technique we propose uses a multi-layered Deconvolutional Network (deconvnet), as proposed by (Zeiler et al., 2011), to project the feature activations back to the input pixel space. We also perform a sensitivity analysis of the classifier output by occluding portions of the input image, revealing which parts of the scene are important for classification.

Using these tools, we start with the architecture of (Krizhevsky et al., 2012) and explore different architectures, discovering ones that outperform their results on ImageNet. We then explore the generalization ability of the model to other datasets, just retraining the softmax classifier on top. As such, this is a form of supervised pre-training, which contrasts with the unsupervised pre-training methods popularized by (Hinton et al., 2006) and others (Bengio et al., 2007; Vincent et al., 2008). The generalization ability of convnet features is also explored in concurrent work by (Donahue et al., 2013).



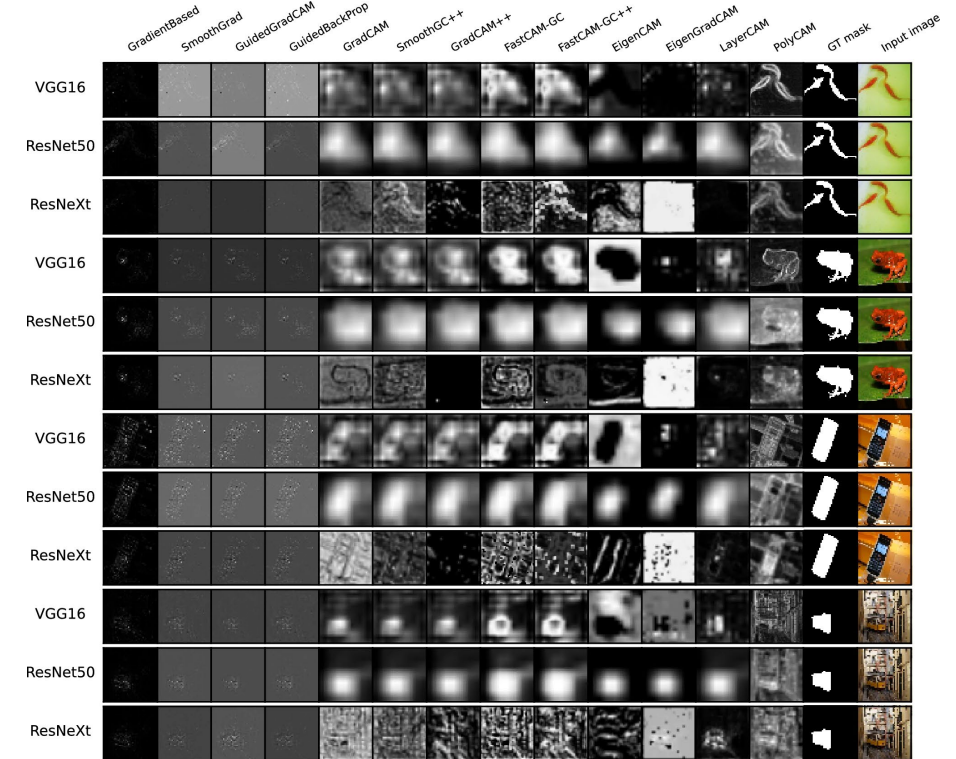
Visualization for machine learning

- Activation maps (or feature maps)
 - The tensors a hidden layer outputs for a given input.
 - *See what patterns fire where.*
 - Different from saliency maps.
 - Saliency maps give information on the most important parts of an image.
 - Class-discriminative saliency maps help distinguish between classes.



Saliency maps

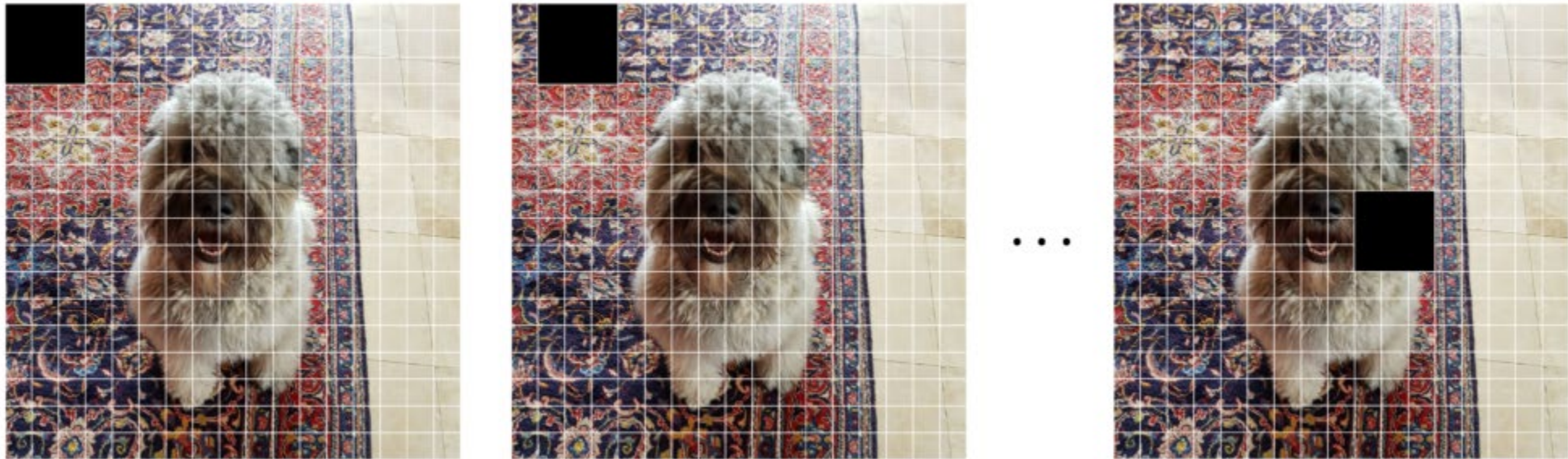
- Saliency maps (a.k.a. sensitivity maps)
- Idea: sensitivity of class to each pixel
- Heatmaps highlight input regions most influential for a model's prediction.
- You have a model $f(x)$ and a target class score $s_c(x)$
 - Which parts of (x) would most change $s_c(x)$ if I nudge them a bit?
 - Compute $\nabla_x s_c(x)$ as a per-feature sensitivity
 - For images: take the per-pixel gradient magnitude $|\nabla_x s_c(x)|$



Saliency map

Find me pixels responsible for the prediction

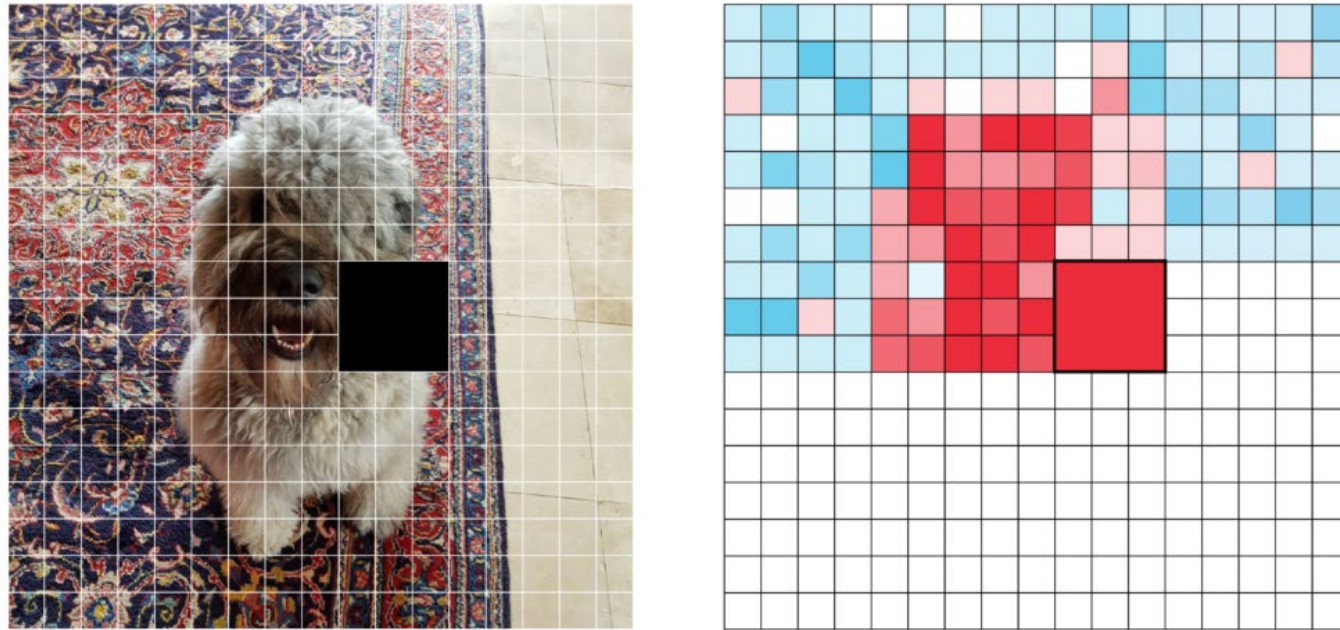
Saliency via occlusion: mask parts of the image before feeding to CNN, check how much predicted probabilities change



Saliency map

Find me pixels responsible for the prediction

Saliency via occlusion: mask parts of the image before feeding to CNN, check how much predicted probabilities change



Saliency map

Find me pixels responsible for the prediction

